NoSQL avec MongoDB

Cours 5 - Schéma JSON de Validation

Steve Lévesque, Tous droits reservés © où applicables

Table des matières

1 Documents Impurs

2 Validation des documents

3 Création du schéma JSON

4 Insertion de documents invalides

Documents impurs

Une règle de pouce de MongoDB est la **simplicité**.

Il est important par conception (design) que la base de données soit simple, rapide et efficace, peu importe en termes d'utilisation ou de performances.

Figure: La simplicité et rapidité de traitement des données.

Documents Impurs

Ceci dit, des désavantages peuvent survenir en lien avec l'intégrité du corpus des données.

Il faut se rappeler que, par défaut, tous les documents peuvent être sauvegardés dans la base de données **peu importe leur composition** vis-à-vis les données actuellement présentes.

Documents Impurs

Cette grande liberté peut causer des problèmes majeurs avec une base de données qui doit maintenir une structure pure à tout coût.

La base de données relationnelle (MySQL, Postgres, etc.) le fait par conception (design), mais elles sont plus fastidieuses à utiliser pour les opérations courantes et simples, ainsi que le développement logiciel (plus de temps à initialiser les aspects divers).

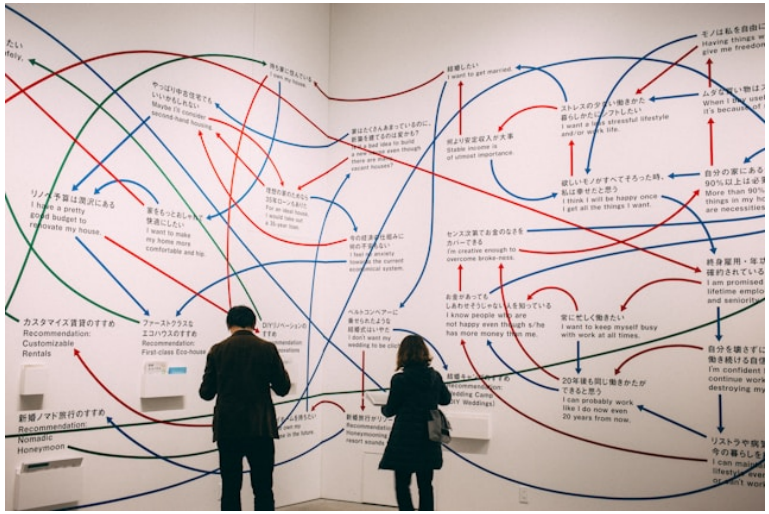
Validation des documents

Pour avoir une certaine assurance contre les documents irréguliers et inconforts au contenu de la base de données, il est possible d'intégrer des règles de validations (valideur) à la collection touchée.

Avec un niveau de validation et d'action complètement personnalisable, les manipulations quelconques sont gérées comme il le faut.

Création du schéma JSON

Le schéma JSON est créé avec des contraintes sur les **propriétés des documents (attributs et leurs contraintes spécifiques)**.



Création du schéma JSON

Listing: https://labex.io/tutorials/mongodb-how-to-use-json-schema-validation-436478

```
# Steve Levesque, All rights reserved where applicable
from pymongo import MongoClient

pokemon_schema = {"$jsonSchema": {
    "title": "Record of a Pokemon",
    "description":
        "This document records the details of a Pokemon",
    "bsonType": 'object',
    "required": ["Name"],
    "properties": {
        "Name": {
            "type": "string",
            "minLength": 2,
}}}}
client = MongoClient()
db = client.PokemonJSONSchema
db.create_collection("PokemonsValidated", validator=pokemon_schema,
                     validationLevel='strict', validationAction='error')
collection = db.PokemonsValidated
```

Insertion de documents invalides

Lorsqu'un document invalide est inséré (modifié), un message d'erreur étant à même titre qu'une erreur d'exécution est déclenché.

Insertion de documents invalides

Listing: Le document 2 sur 3 est invalide puisqu'il ne respecte pas la règle de validation du schéma JSON

```
# Steve Levesque, All rights reserved
from pymongo import MongoClient

client = MongoClient()
db = client.PokemonJSONSchema
collection = db.PokemonsValidated

collection.insert_one({"#": 0, "Name": "Valid"}) # Valid document.
collection.insert_one({"#": 0, "Name": "T"}) # Error here.
collection.insert_one({"#": 0, "Name": "ABC"}) # Not reached.

"""
pymongo.errors.WriteError: Document failed validation, full error:
...
[{'operatorName': 'minLength', 'specifiedAs': {'minLength': 2}, 'reason':
'specified string length was not satisfied', 'consideredValue': 'T'}]}]}]}}}

Process finished with exit code 1
"""
```

Insertion de documents invalides

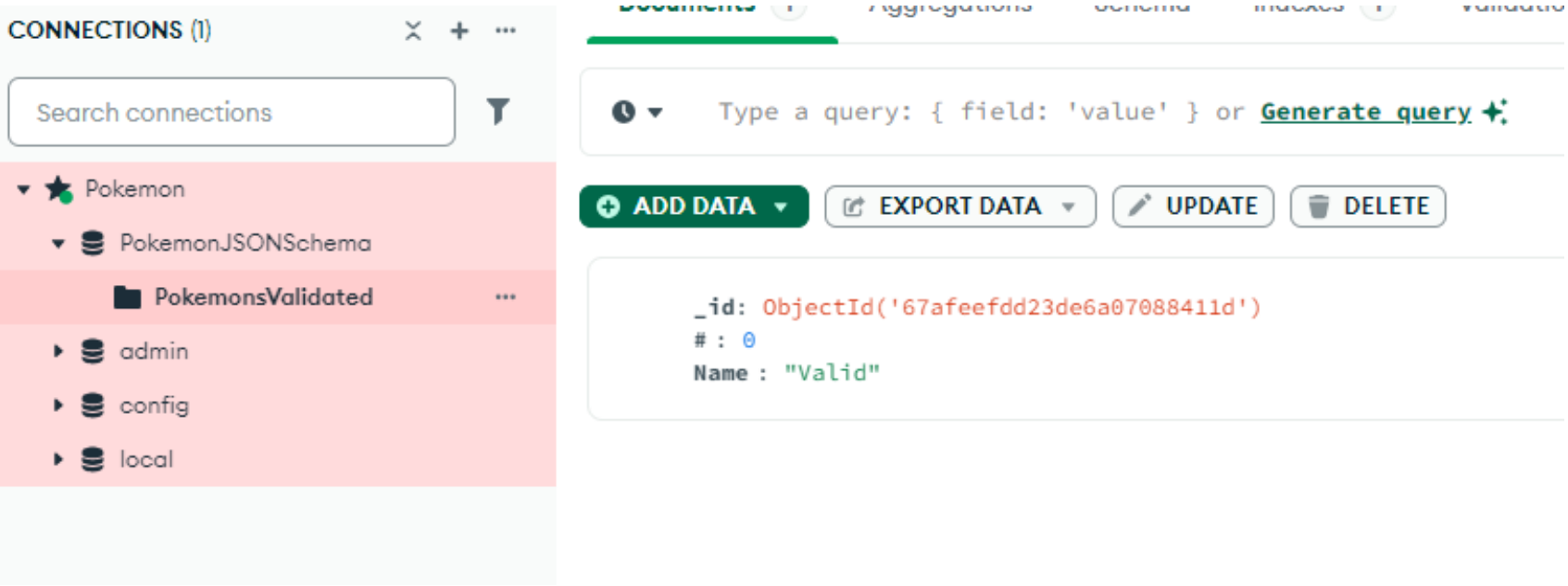


Figure: Seulement le 1er document est dans la base de données, puisque le 2e document est invalide, et l'erreur produite empêche l'insertion du 3e document.

Bibliographie

- https://labex.io/tutorials/mongodb-how-to-use-json-schema-validation-436478